

DATA SCIENCE & ANALYSIS

The Ultimate Pandas Handbook

Master Data Manipulation & Analysis

Author

RahulSinghShah

NAVIGATE YOUR LEARNING

Table of Contents

01

Core Datatypes

Introduction to Series and DataFrames

02

Importing and Exporting Data

Handling CSVs safely into and out of Pandas

03

Describing & Viewing Data

Using attributes like dtypes, columns, and describe

04

Selecting & Filtering: .loc vs .iloc

The correct way to extract subsets of data

05

Cleaning Data

Dealing with incorrect formatting and missing NaNs

06

Feature Engineering

Creating new columns from existing data

07

Data Visualization

Quick plots straight from your DataFrame

1. Core Datatypes

Pandas is the workhorse of Data Science. If NumPy provides the mathematical engine, Pandas provides the Excel-like tabular structure that we need to clean and analyze real-world data safely.

Pandas has two primary data structures:

The Series (1-Dimensional)

A Series is essentially a single column of data. It works like a Python list or a 1D NumPy array, but comes with an interactive index.

```
import pandas as pd

colours = pd.Series(["Blue", "Black", "Red"])
print(colours)
# 0      Blue
# 1     Black
# 2       Red
# dtype: object
```

The DataFrame (2-Dimensional)

A DataFrame is a 2-Dimensional table containing rows and columns. Think of it as a dictionary of Series objects merged together. It's the equivalent of an Excel spreadsheet.

```
car_data = pd.DataFrame({"Car make": ["Toyota", "BMW"],  
                          "Colour": ["Blue", "Black"]})
```

2. Importing and Exporting Data

In the real world, you don't type datasets by hand. You import them from files or databases. The most common format in data science is `.csv` (Comma Separated Values).

Importing

When reading a file, Pandas automatically parses headers and guesses data types.

```
car_sales = pd.read_csv("car-sales.csv")
```

Exporting

After you've cleaned or analyzed data, you usually want to save it back to a file.

```
# Save DataFrame to a new CSV file
# (index=False prevents Pandas from exporting the numerical
index as a new column)
car_sales.to_csv("exported-car-sales.csv", index=False)
```

3. Describing & Viewing Data

Once data is imported, you must understand what you are looking at before analyzing it. Use **Attributes** (which don't have brackets `()`) and **Methods** (which do).

Core Attributes

Attributes give you metadata *about* the DataFrame without modifying or calculating anything.

```
print(car_sales.dtypes)    # Shows the data type of every
                           column
print(car_sales.columns)   # Returns a list of column names
print(car_sales.index)     # Returns the index range (e.g.,
                           0 to 10)
```

Core Methods

Methods actively calculate or fetch information.

```
car_sales.head(10)      # View the first 10 rows
car_sales.tail()        # View the last 5 rows

# Generate statistics (mean, min, max, percentiles) for
numerical columns
car_sales.describe()
```

4. Selecting & Filtering: .loc vs .iloc

Extracting specific rows and columns is how we query our data.

Filtering Data

You can pass a condition directly into the DataFrame brackets to isolate specific rows.

```
# Return all rows where the "Make" column is exactly  
"Toyota"  
toyota_cars = car_sales[car_sales["Make"] == "Toyota"]
```

.loc vs .iloc

These are Pandas' dedicated slicing tools. It is critical to know the difference!

- **.loc** refers to the **Location** (based on the index *label*).
- **.iloc** refers to the **Index Location** (based on the absolute numerical *position*, exactly like a Python list).


```
animals = pd.Series(["dog", "cat", "snake", "horse",  
                    "tiger"],  
                    index=[0, 3, 4, 7, 3])  
  
# .loc looks at the LABEL.  
# There are two items with the label '3'  
print(animals.loc[3])  
# 3      cat  
# 3     tiger  
  
# .iloc looks at the strict absolute POSITION.  
# The 3rd item (0-indexed) is "horse"  
print(animals.iloc[3])  
# horse
```

5. Cleaning Data

Real-world data is extremely messy. It will feature string formatting errors and missing values.

Formatting Errors

Sometimes numeric data comes with string symbols like `$`, `,`, or `.` attached. Before Pandas can do math on it, we must strip those symbols and convert the column to an integer.

```
# Replace regex patterns ($ , .) with nothing, then cast to
int
car_sales["Price"] = (
    car_sales["Price"]
    .str.replace('[\$\,\.]', '', regex=True)
    .astype(int)
)
```

Handling Missing Data (NaNs)

When a CSV has a blank cell, Pandas loads it as `NaN` (Not a Number). You have two choices: drop the missing data or fill it.

```
csm = pd.read_csv("car-sales-missing-data.csv")

# Ensure column is numeric before doing math
csm["Odometer"] = pd.to_numeric(csm["Odometer"],
errors='coerce')

# Choice 1: Fill missing values with the mean/average
mean_val = csm["Odometer"].mean()
csm["Odometer"].fillna(mean_val, inplace=True)

# Choice 2: Drop any rows that are missing data
csm_clean = csm.dropna()
```

6. Feature Engineering

Creating new columns from existing data is called Feature Engineering. It's how we expose new patterns to Machine Learning models.

Creating Columns

You can declare a new column simply by assigning values to a column name that doesn't exist yet.

```
# Create a column from a list (list length must perfectly
match DataFrame length)
fuel_economy = [7, 8, 7, 9, 8, 7, 9, 7, 8, 9]
car_sales["Fuel per 100KM"] = fuel_economy

# Create a column using math from other columns
car_sales["Total Fuel Used"] = (
    car_sales["Odometer (KM)"] / 100 * car_sales["Fuel per
100KM"]
)

# Create a column filled with a single static value
car_sales["Wheels"] = 4
```

Removing Columns

If you make a mistake or no longer need a column, drop it using `axis=1`.

```
# axis=1 tells Pandas to drop the COLUMN named "Wheels".  
(axis=0 drops rows)  
car_sales = car_sales.drop("Wheels", axis=1)
```

7. Data Visualization

Pandas is directly hooked up to `matplotlib`, meaning you don't have to export data to create basic charts. You can plot data directly from the DataFrame.

```
import matplotlib.pyplot as plt

# A quick line graph showing the fluctuation of odometer
readings
car_sales["Odometer (KM)"].plot()

# A bar chart showing the frequency of total rooms in a
housing dataset
housing_data.total_rooms.value_counts().plot(kind="bar")
```

When doing exploratory analysis, these instant plots are invaluable for understanding data distribution at a glance.



Complete!



You've mastered the fundamentals of **Pandas**.

PANDAS HANDBOOK

by RahulSinghShah